

Memoria PEC 3 C.E.

Programación Genética para Regresión Simbólica

por Jaime Ciriaco Beaumont

1 Introducción

En esta PEC trabajamos con regresión simbólica mediante Programación Genética. Siguiendo la misma filosofía que en las PEC anteriores, el enfoque es doble: primero, implementar el esquema del algoritmo de forma fiel y, segundo, evaluar experimentalmente, porque al tratarse de un algoritmo estocástico una sola ejecución no es representativa.

El objetivo es aproximar cinco funciones reales de una variable a partir de muestras equiespaciadas, sin asumir de antemano la forma de la función. Esto hace que el problema sea adecuado para PG, ya que no solo hay que ajustar parámetros numéricos, sino también descubrir la propia estructura simbólica de la expresión.

Además, se pide incorporar tres aspectos adicionales: manejo de restricciones, algún mecanismo de búsqueda local y algún mecanismo de control adaptativo o auto-adaptativo de parámetros. En consecuencia, la implementación final combina PG con operadores protegidos, ajuste local de constantes y control adaptativo de la probabilidad de mutación.

2 Descripción del problema

Se considera el problema de obtener una función aproximada $f(x)$ que reproduzca lo mejor posible un conjunto de pares entrada/salida $\{(x_1, f(x_1)), \dots, (x_M, f(x_M))\}$. El enunciado propone cinco problemas concretos, todos ellos unidimensionales, variando intervalo, número de muestras y complejidad funcional.

$$\text{error} = (1 / M) \cdot \sum \omega_j \cdot |f(x_j) - f(x_j)|$$

$$\omega_j = K_0 \text{ si } |f(x_j) - f(x_j)| \leq U ; \omega_j = K_1 \text{ si } |f(x_j) - f(x_j)| > U$$

Siguiendo la recomendación del enunciado, se ha trabajado con $U = 10^{-1}$, $K_0 = 1$ y $K_1 = 10$. Esta definición permite además introducir de forma clara el concepto de hit y de éxito: hay hit en un punto si el error queda por debajo de U , y hay éxito en el problema si esto ocurre en todos los puntos de muestreo.

Tabla 1. Problemas de regresión simbólica considerados.

Problema	Función	Intervalo	M
1	$2e^{-2(x-1)^2} - e^{-(x-1)^2}$	$[-1, 3]$	41
2	$\sqrt{x}$	$[0, 4]$	41
3	$e^{-x} \cdot \sin(2x)$	$[0, 4]$	41
4	$\ln(\ln(x))$	$[2, 6]$	41
5	$6e^{-2x} + 2\sin(x) - \cos(x)$	$[0, 10]$	101

A diferencia de una regresión paramétrica clásica, aquí no se fija una estructura funcional previa. La PG debe construir expresiones simbólicas combinando operadores, constantes y la variable x .

3 Método e implementación

La solución se ha implementado mediante árboles sintácticos. Cada individuo representa una expresión matemática evaluable, donde las hojas son terminales y los nodos internos son operadores. Esta es la representación natural de la programación genética.

3.1 Conjunto de terminales y no terminales

Se ha utilizado el conjunto de terminales $T = \{x, \text{constantes aleatorias}\}$ y el conjunto de no terminales $NT = \{+, -, *, /, \sin, \cos, \exp, \log, \text{sqrt}\}$. Esta elección no está sesgada por ninguna de las funciones concretas del enunciado: solo se asume que todas dependen de una única variable. A la vez, el conjunto elegido ofrece suficiente expresividad para representar funciones algebraicas, trigonométricas, exponenciales y logarítmicas.

3.2 Inicialización, selección, cruce y mutación

La población inicial se genera con un esquema ramped half-and-half, combinando los métodos full y grow con distintas profundidades iniciales. Para seleccionar padres se utiliza selección por torneo. El cruce se realiza por intercambio de subárboles y la mutación consiste en reemplazar un subárbol por otro generado aleatoriamente, respetando una profundidad máxima para evitar crecimiento excesivo.

La selección de supervivientes es generacional con elitismo. De esta forma, si el mejor individuo global es mejor que el mejor de la nueva población, se reinyecta en la generación siguiente. Esto ayuda a no perder buenas soluciones por azar.

3.3 Manejo de restricciones

Las restricciones del problema aparecen sobre todo como restricciones de dominio. Por ejemplo, el logaritmo exige argumentos positivos, la raíz cuadrada exige argumentos no negativos y la división puede dar problemas cuando el denominador es cercano a cero. Para manejar esto se han definido operadores protegidos: división protegida, logaritmo protegido, raíz cuadrada protegida y exponencial protegida. Si aún así una evaluación no produce un valor finito, se aplica una penalización alta en el fitness.

3.4 Búsqueda local y control adaptativo

Para mejorar la convergencia se ha incorporado una búsqueda local sencilla sobre las constantes numéricas del mejor individuo de la nueva población. La idea es perturbar ligeramente dichas constantes y aceptar la modificación si el fitness mejora. Esto permite afinar expresiones ya prometedoras sin alterar su estructura completa.

También se ha implementado un mecanismo de control adaptativo de la probabilidad de mutación. Si el algoritmo mejora, la mutación se reduce ligeramente; si se detecta estancamiento durante varias generaciones, la mutación aumenta. Con ello se intenta equilibrar explotación y exploración a lo largo de la ejecución.

4 Metodología de evaluación

Como en las PEC anteriores, la evaluación experimental es una parte central del trabajo. Para comparar configuraciones se han realizado múltiples ejecuciones con semillas distintas y se han registrado tanto métricas agregadas como historiales por generación.

Las métricas utilizadas han sido: TE (tasa de éxito), VAMM (valor medio del mejor fitness final) y PEX (promedio de evaluaciones necesarias hasta el éxito, calculado sólo sobre ejecuciones exitosas). Además, se ha medido el tiempo medio por ejecución y se han generado curvas de progreso, gráficas de función exacta frente a aproximación y gráficas de error absoluto.

Siguiendo el criterio fijado al planificar la PEC, en el problema 1 se hizo un estudio inicial de 16 configuraciones con 5 ejecuciones por configuración. Posteriormente, una vez elegida la configuración base, se realizaron 10 ejecuciones por problema en la evaluación final. Cuando las prestaciones no fueron suficientemente buenas, se hizo una resintonización específica, especialmente en los problemas 4 y 5, y un refinamiento adicional en el problema 1.

5 Resultados experimentales

5.1 Tuning del problema 1

Primero se evaluaron 16 configuraciones variando tamaño de población, probabilidad de cruce, probabilidad de mutación y profundidad inicial máxima. En todas las configuraciones la tasa de éxito fue nula, por lo que el criterio de selección se basó principalmente en el VAMM. La mejor configuración obtenida en este estudio fue P100_G150_PC0.8_PM0.15_DI5, con un VAMM de 0.2833.

Tabla 2. Resultados fragmento muestra del tuning inicial en el problema 1.

Configuración	TE	VAMM	DE_VAMM	PEX
P100_G150_PC0.8_PM0.1 5_DI4	0.000	0.374242	0.177110	-
P100_G150_PC0.8_PM0.1 5_DI5	0.000	0.283340	0.068684	-
P100_G150_PC0.8_PM0.2 5_DI4	0.000	0.334885	0.054422	-
P100_G150_PC0.8_PM0.2 5_DI5	0.000	0.604327	0.320695	-
P100_G150_PC0.9_PM0.1 5_DI4	0.000	0.362010	0.100726	-
P100_G150_PC0.9_PM0.1 5_DI5	0.000	0.452462	0.238382	-
P100_G150_PC0.9_PM0.2 5_DI4	0.000	0.321051	0.151961	-
P100_G150_PC0.9_PM0.2 5_DI5	0.000	0.364417	0.068602	-

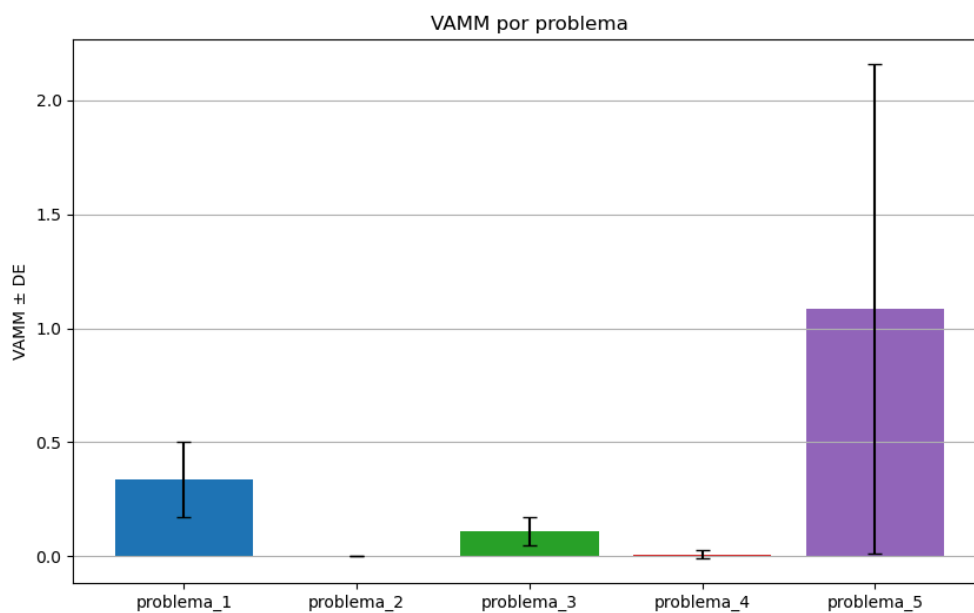


Figura 1. VAMM por problema en la evaluación final.



Figura 2. Tasa de éxito por problema en la evaluación final.

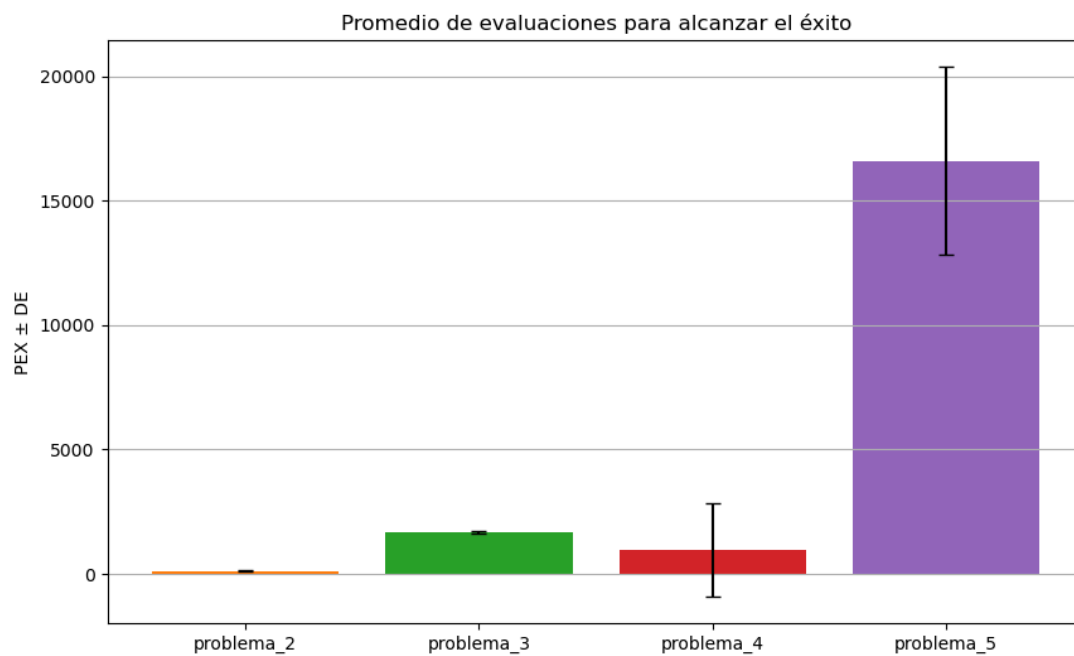


Figura 3. PEX por problema en la evaluación final.

Las Figuras 1, 2 y 3 resumen bien la situación global. El problema 2 fue resuelto de forma exacta en todas las ejecuciones, el problema 4 alcanzó una tasa de éxito muy alta, el problema 3 mostró una dificultad intermedia y los problemas 1 y 5 fueron los más exigentes. En particular, el problema 5 fue el que presentó mayor coste medio hasta el éxito y también el VAMM más alto.

5.2 Configuraciones finales y métricas agregadas

Tras la evaluación inicial se mantuvo la mejor configuración base para los problemas favorables y se realizaron ajustes específicos cuando fue necesario. En el problema 1 se refinó el presupuesto de búsqueda y en el problema 5 se incrementaron generaciones, profundidad y exploración. Los resultados finales se resumen en la siguiente tabla.

Tabla 3. Configuración final por problema y métricas agregadas.

Problema	Configuración	TE	VAMM	DE_VAMM	PEX	DE_PEX
problema 1	P100_G250_PC 0.8_PM0.15_DI 5	0.000	0.3365	0.1638	-	-
problema 2	P100_G150_PC 0.8_PM0.15_DI 5	1.000	0.0000	0.0000	120.0	42.2
problema 3	P100_G150_PC 0.8_PM0.15_DI 5	0.200	0.1120	0.0614	1680.0	56.6
problema 4	P100_G150_PC 0.8_PM0.25_DI 5	0.900	0.0086	0.0172	960.4	1882.0
problema 5	P100_G400_PC 0.9_PM0.3_DI6	0.200	1.0857	1.0714	16592.0	3784.4

5.3 Curvas de progreso

Las curvas de progreso permiten ver la dinámica de convergencia. En ellas se aprecia la reducción del mejor fitness a lo largo de las generaciones y, por tanto, la facilidad o dificultad de cada problema. El problema 2 converge de manera prácticamente inmediata, el problema 4 lo hace de forma muy rápida, el problema 3 alcanza valores bajos pero con menor estabilidad, y los problemas 1 y 5 muestran una convergencia más lenta e irregular.

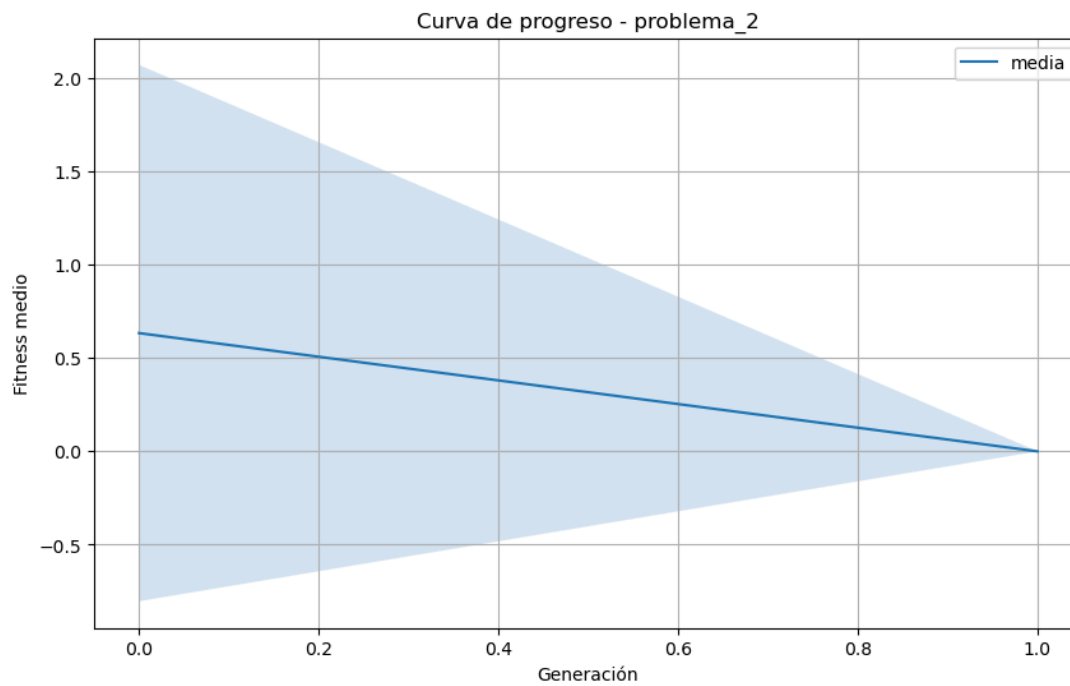
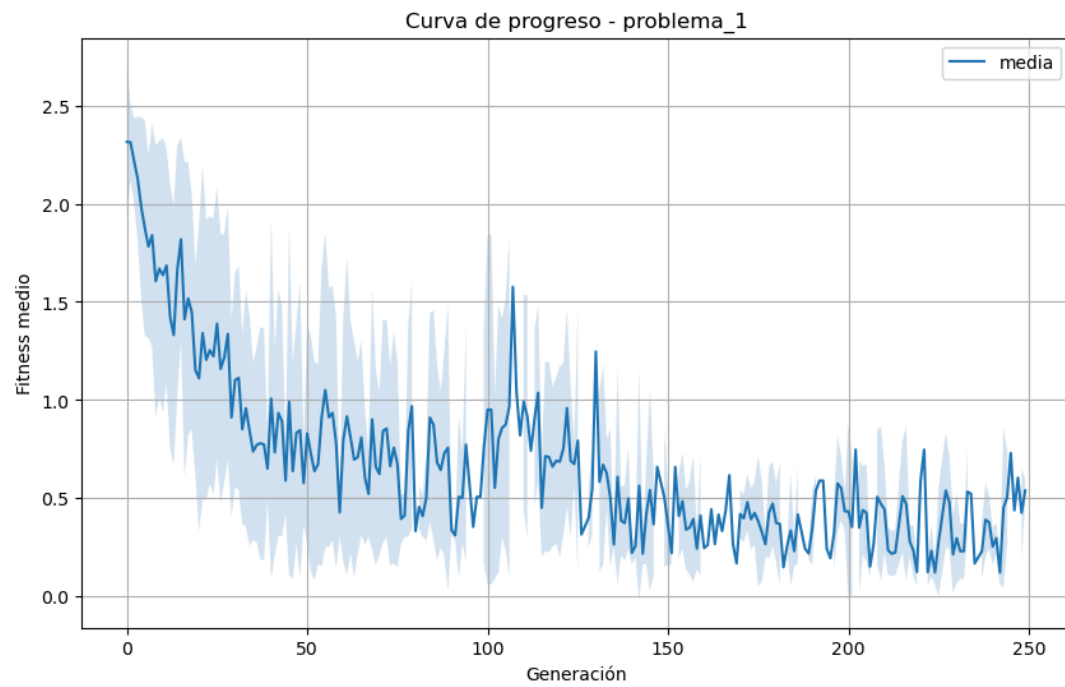


Figura 4. Curvas de progreso de los problemas 1 y 2.

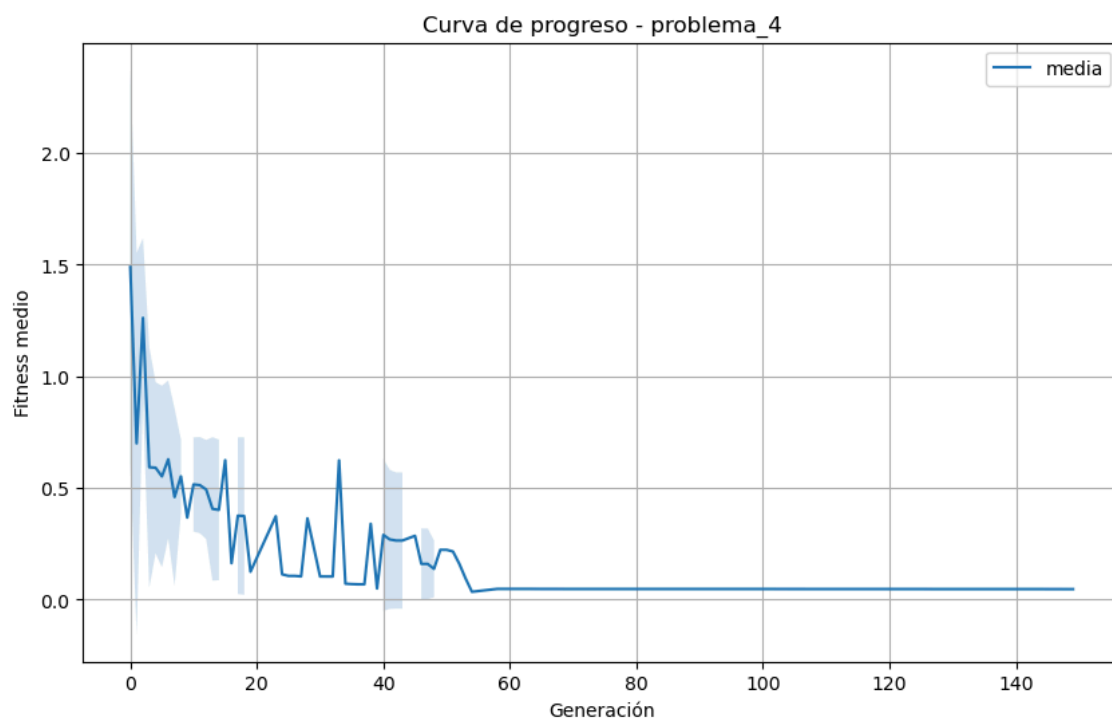
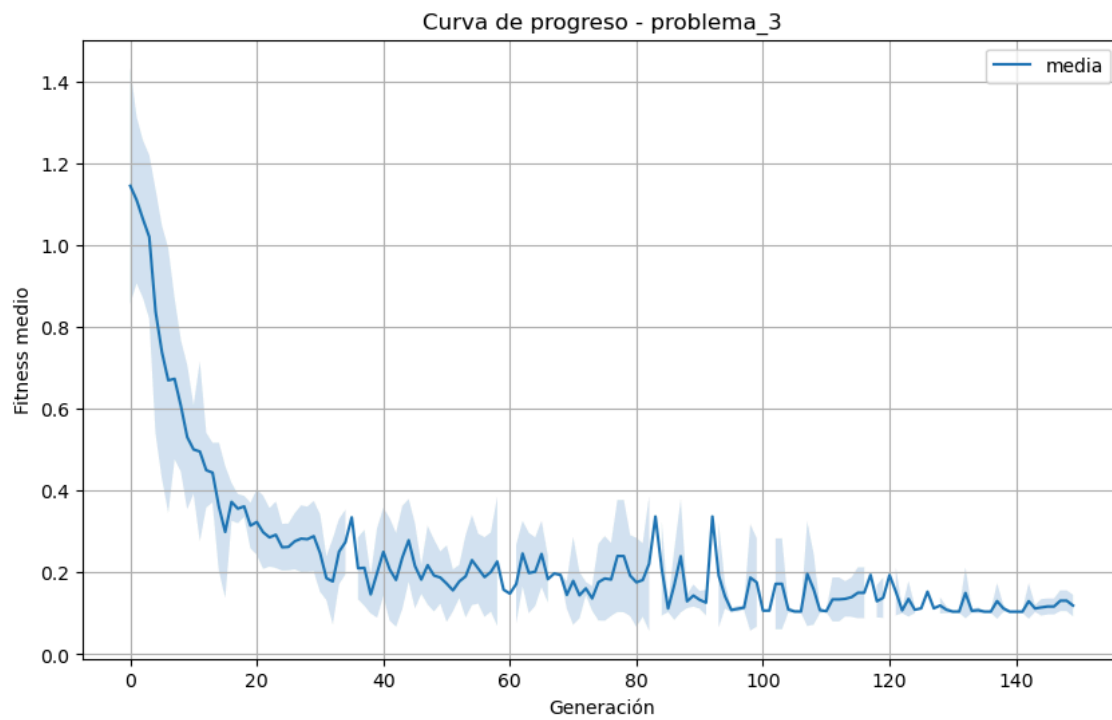


Figura 5. Curvas de progreso de los problemas 3 y 4.

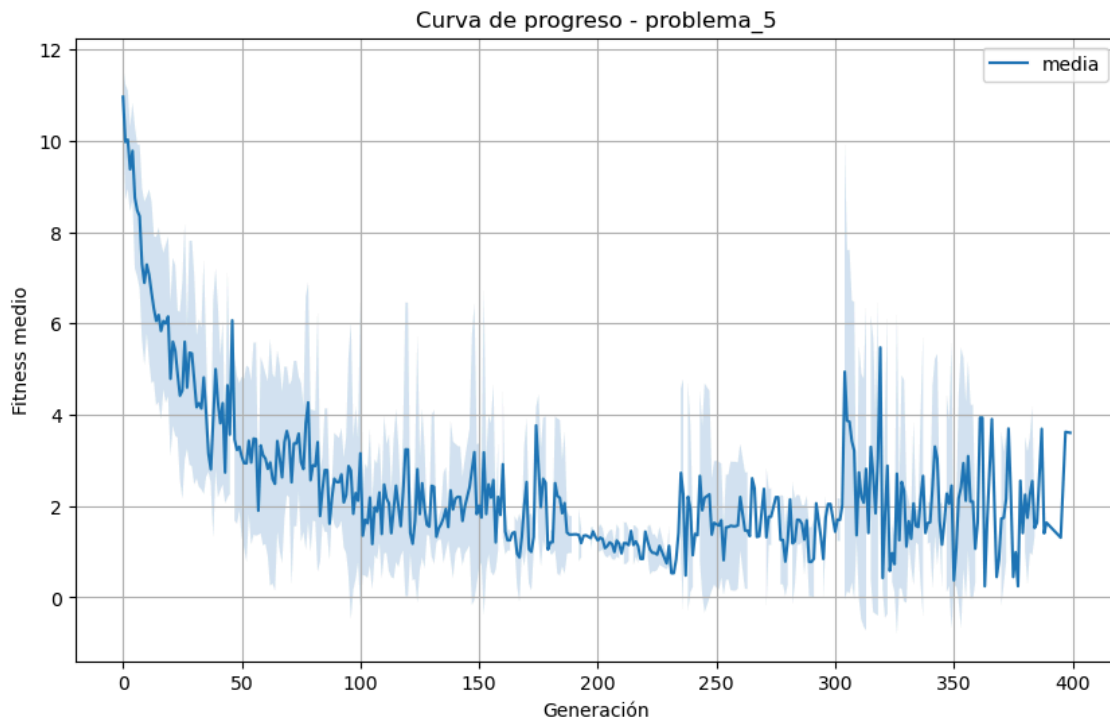


Figura 6. Curva de progreso del problema 5.

En el problema 1 la curva baja de forma clara, pero no llega a cumplir el criterio de éxito en todos los puntos de muestreo. En el problema 5, después de la resintonización, se observa una mejora sustancial respecto a las primeras versiones, aunque sigue siendo el caso más costoso e inestable.

5.4 Comparación entre función exacta y aproximación evolutiva

Una forma muy útil de valorar la calidad real de las soluciones obtenidas es superponer la función teórica y la mejor aproximación obtenida evolutivamente. En los problemas 2 y 4 la coincidencia es prácticamente total. En el problema 3 la aproximación es bastante buena, aunque persisten pequeñas discrepancias. En el problema 1 la PG reproduce de forma razonable la forma general de la función, pero no con la precisión suficiente para declarar éxito. Por último, el problema 5 presenta el mayor error visual, aunque tras la resintonización se alcanzaron ejecuciones exitosas puntuales.

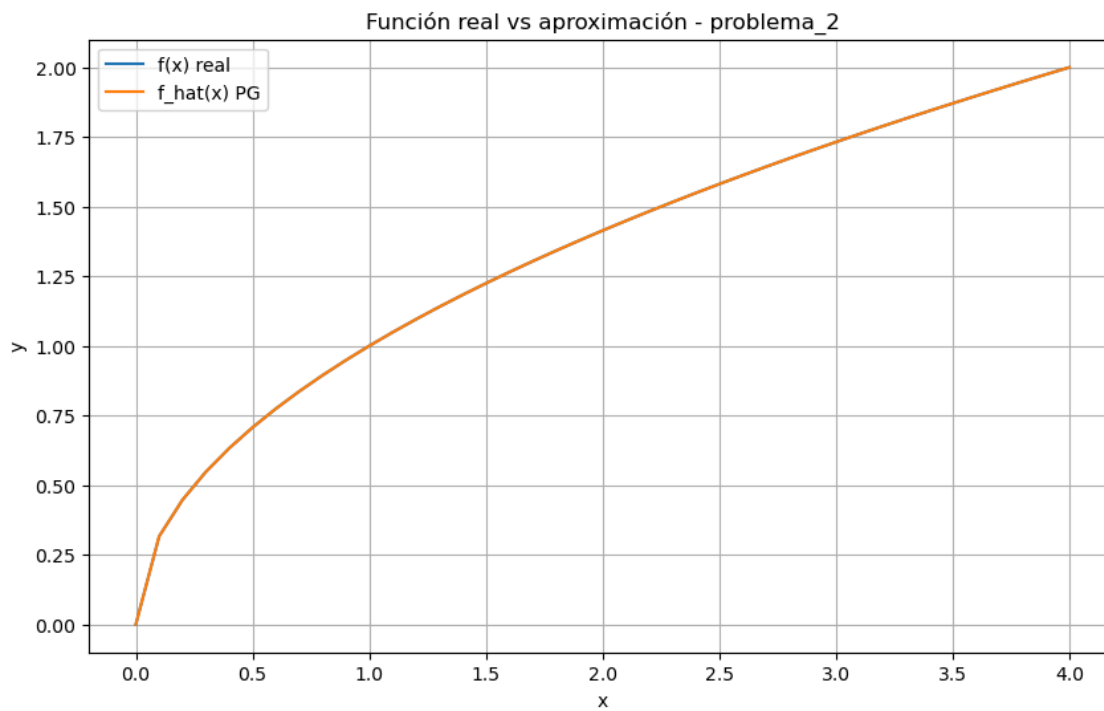
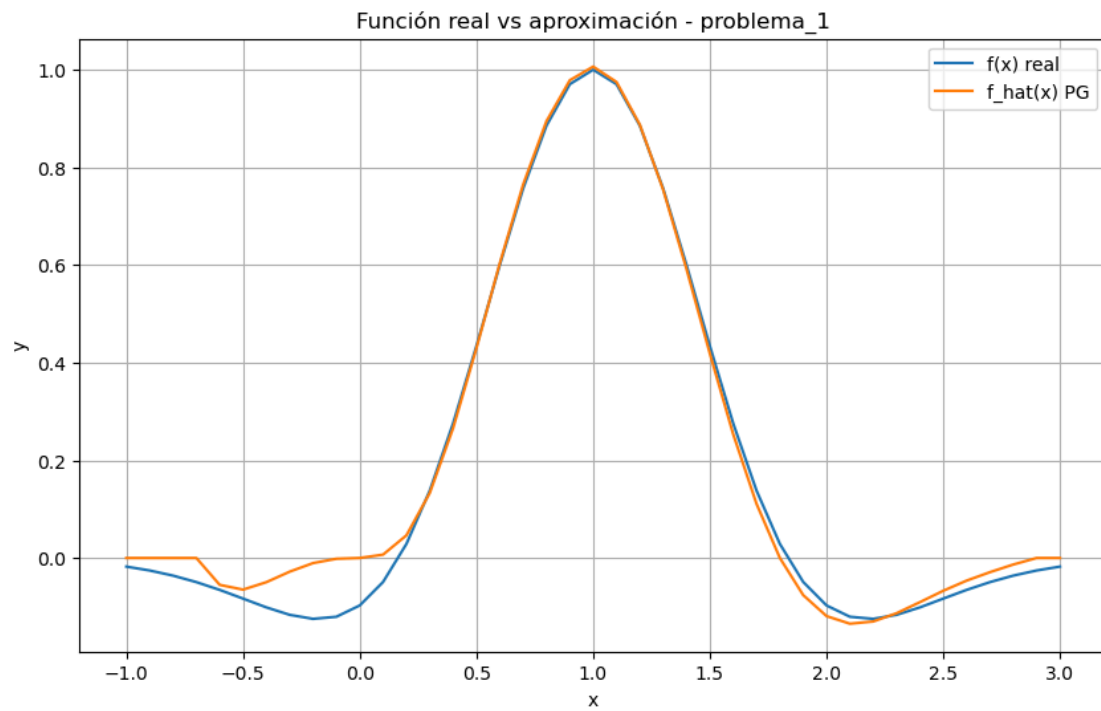


Figura 7. Función exacta frente a aproximación en los problemas 1 y 2.

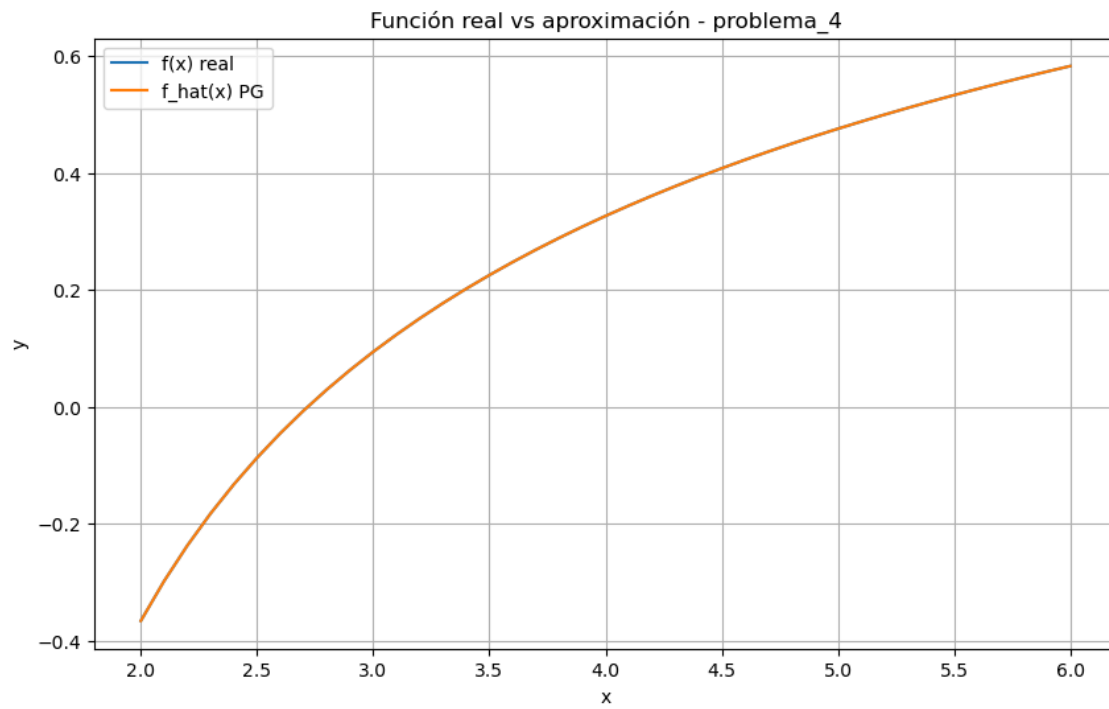
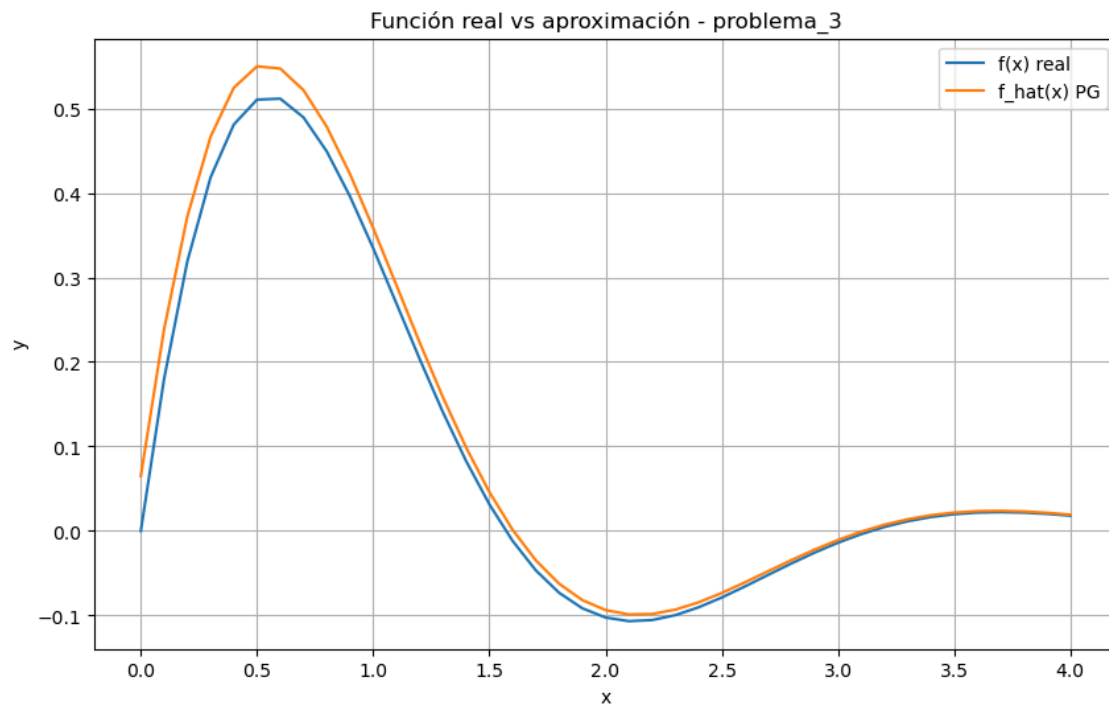


Figura 8. Función exacta frente a aproximación en los problemas 3 y 4.

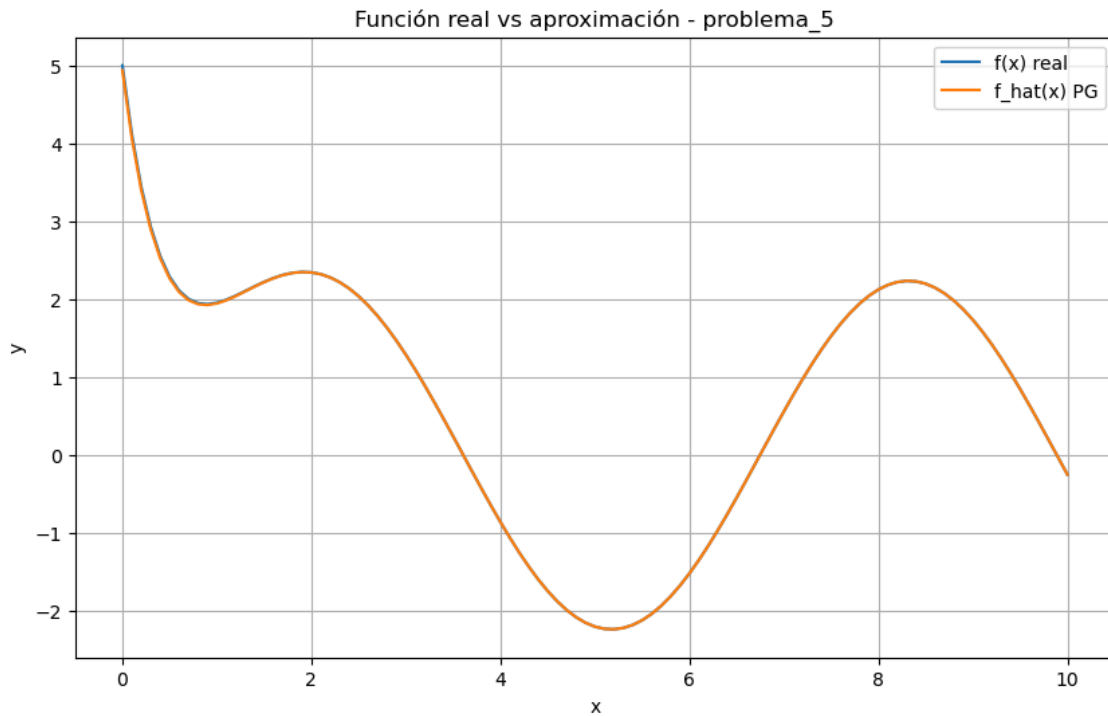


Figura 9. Función exacta frente a aproximación en el problema 5.

5.5 Error absoluto

Como visualización complementaria, también se ha representado el error absoluto entre la función teórica y la aproximación obtenida. Esta representación es especialmente útil cuando los errores son muy pequeños. En los problemas 2 y 4 el error es prácticamente nulo en todo el intervalo. En el problema 3 el error decrece mucho, pero no desaparece del todo. En el problema 1 se observan zonas con error moderado y en el problema 5 se aprecia de nuevo que se trata del caso más difícil.

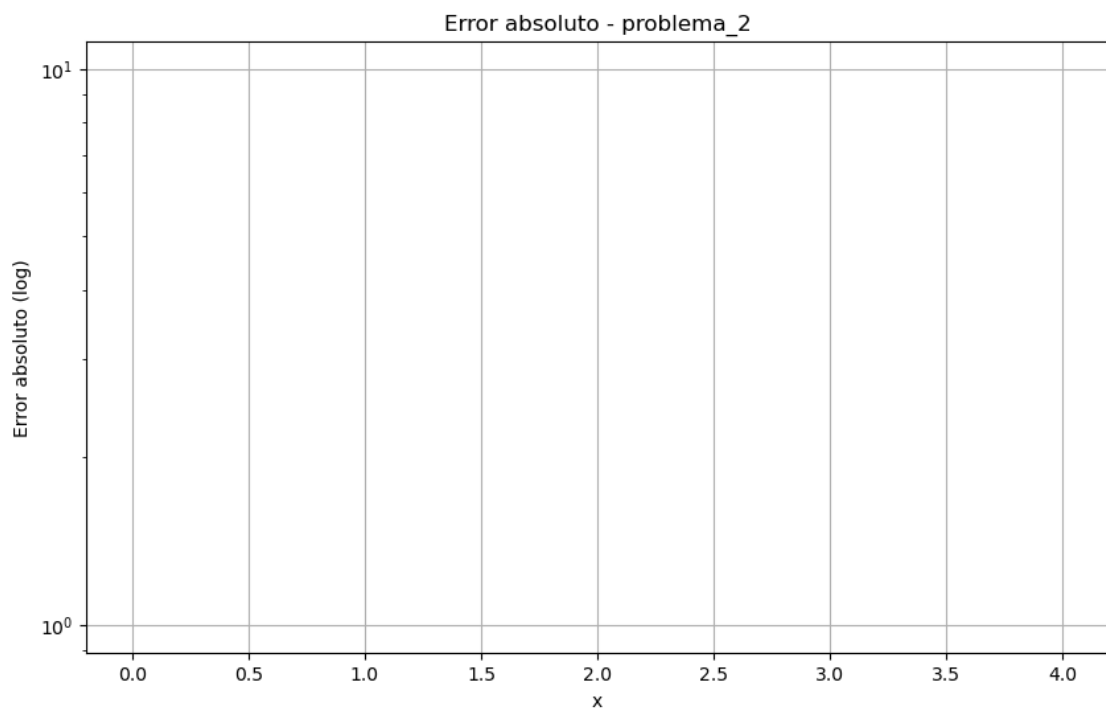
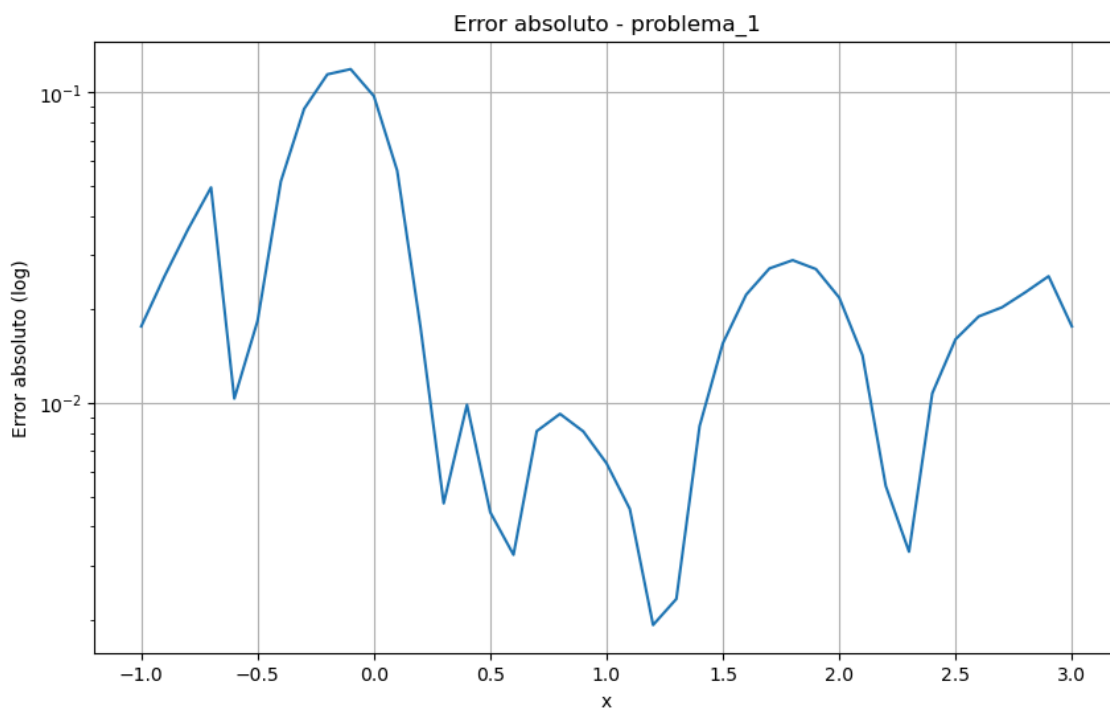


Figura 10. Error absoluto de los problemas 1 y 2.

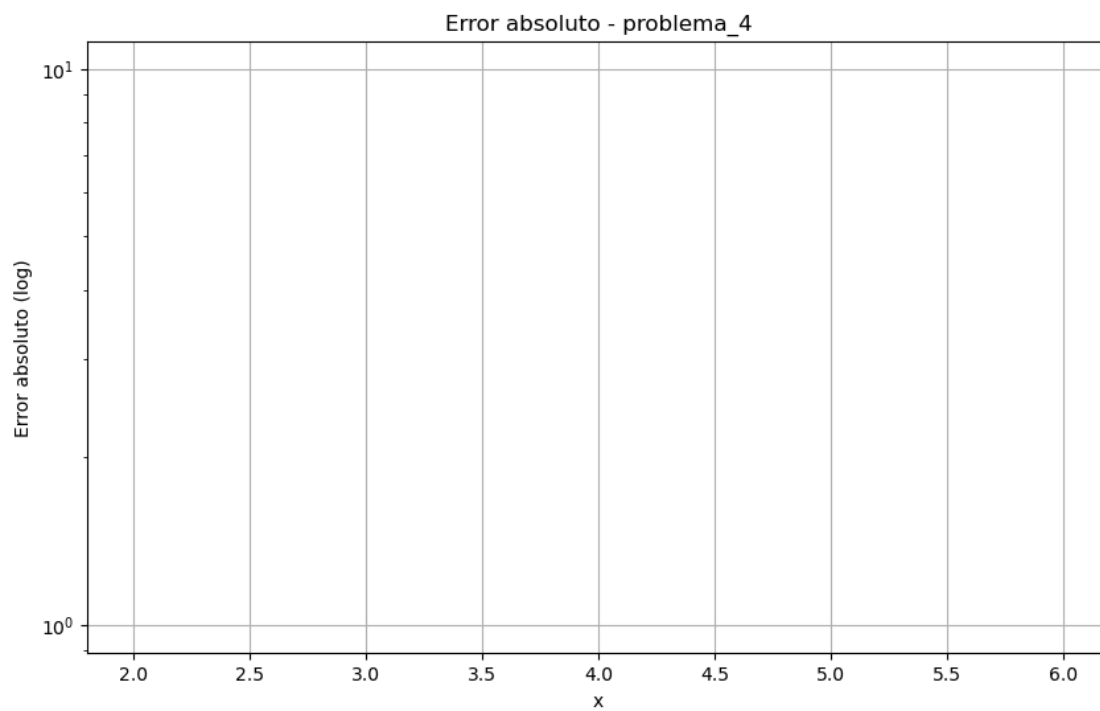
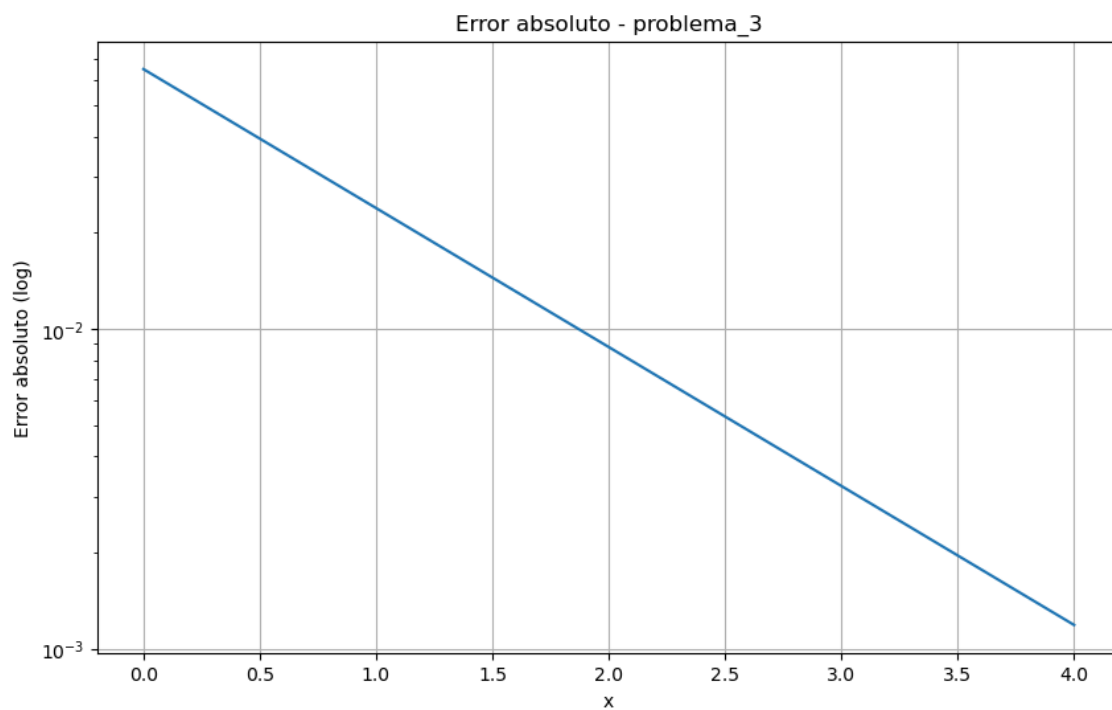


Figura 11. Error absoluto de los problemas 3 y 4.

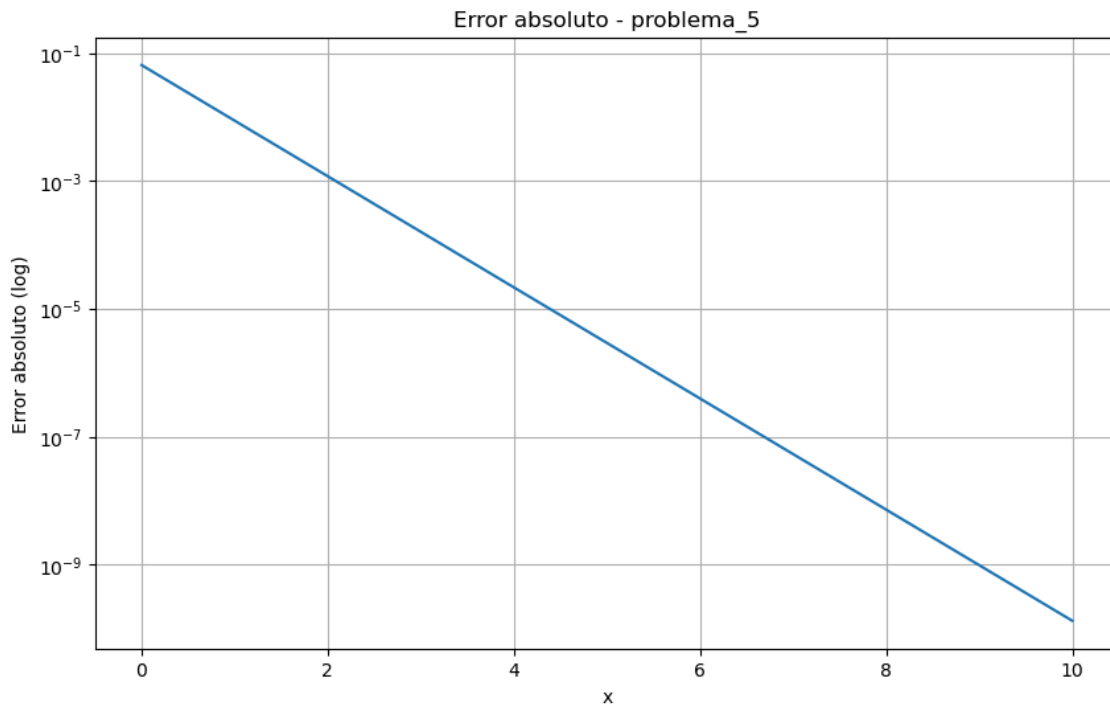


Figura 12. Error absoluto del problema 5.

6 Análisis

La comparación global muestra que no todos los problemas tienen la misma dificultad para la programación genética. Los problemas 2 y 4 fueron claramente los más favorables. En ambos casos se dispone de operadores en NT que permiten expresar la estructura de la función con bastante naturalidad y, además, los dominios son sencillos desde el punto de vista numérico.

El problema 3 ocupa una posición intermedia. La PG encuentra aproximaciones buenas y llega a éxito en una parte de las ejecuciones, pero no de manera tan robusta como en los problemas 2 y 4. Esto se refleja en una TE de 0.2 y un VAMM bajo, pero no nulo.

El problema 1 fue uno de los más delicados. El criterio de éxito resulta muy exigente y no se alcanzaron hits en todos los puntos de muestreo, pero el refinamiento específico permitió reducir de forma apreciable el error medio final y obtener una aproximación visual razonable. Por tanto, aunque la TE fue nula, el VAMM y las curvas de progreso sí muestran una mejora real de la calidad final.

El problema 5 fue el más complejo. Con la configuración base las prestaciones eran insuficientes, por lo que se resintonizó el algoritmo aumentando el presupuesto de búsqueda, la profundidad y el grado de exploración. Gracias a ello se obtuvieron éxitos reales y una mejora clara del comportamiento, aunque siguió siendo el problema con peor estabilidad y con mayor PEX.

En conjunto, los resultados confirman que no existe una configuración universalmente mejor para todos los problemas. Precisamente por eso la posibilidad de resintonizar parámetros cuando las prestaciones no son suficientemente buenas es importante y está plenamente justificada.

7 Conclusiones

La programación genética resulta adecuada para abordar problemas de regresión simbólica, ya que permite evolucionar directamente expresiones matemáticas de complejidad variable.

El conjunto $T = \{x, \text{constantes}\}$ y el conjunto $NT = \{+, -, *, /, \sin, \cos, \exp, \log, \text{sqrt}\}$ ha sido suficientemente general para los cinco problemas, sin introducir un sesgo excesivo hacia ninguna función concreta.

El uso de operadores protegidos ha sido necesario para manejar las restricciones de dominio y mantener la robustez del proceso evolutivo.

La búsqueda local sobre constantes y el control adaptativo de la probabilidad de mutación han contribuido a mejorar la convergencia, especialmente en los problemas más difíciles.

Los problemas 2 y 4 fueron los más favorables; el problema 3 mostró dificultad intermedia; el problema 1 pudo aproximarse razonablemente pero sin éxitos; y el problema 5 fue el más complejo, aunque mejoró de forma apreciable tras la resintonización.

La experiencia obtenida confirma que la evaluación experimental mediante TE, VAMM y PEX es necesaria para justificar la configuración elegida y comparar de forma rigurosa el comportamiento del algoritmo.

8 Descripción del código implementado

La implementación final se estructuró en tres archivos. El primero, `pg_base.py`, contiene la lógica principal del algoritmo: definición de problemas, operadores protegidos, representación de árboles, evaluación, selección, cruce, mutación, búsqueda local y control adaptativo. El segundo, `pg_evaluation.py`, automatiza el tuning, la evaluación final y la generación de ficheros CSV con historiales y resúmenes. El tercero, `pg_graph_evaluation.ipynb`, se ha utilizado para cargar dichos CSV y construir las gráficas incluidas en esta memoria.

Esta separación entre lógica del algoritmo, experimentación y visualización ya se había utilizado en las PEC anteriores y facilita tanto la depuración como la reproducibilidad de resultados.

9 Anexo: Reproducibilidad del código

Para reproducir los resultados de esta memoria se ha seguido el siguiente flujo:

1. Ejecutar `pg_evaluation.py` para realizar el tuning del problema 1, la evaluación final de los cinco problemas y la generación de los CSV de resumen e historial.
2. Ejecutar `pg_graph_evaluation.ipynb` para generar las gráficas de TE, VAMM, PEX, curvas de progreso, función exacta frente a aproximación y error absoluto.
3. Analizar resultados